

**SIMATS ENGINEERING
ASSESSMENT TOOL 1**

COURSE CODE: CSA1410

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks:50

Mode: In Class

Hours : 60 mts

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

I CH.V. Ramayogi / 192424319 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: CH.V. Yogi

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

Annexure A

ANALYTICAL PROBLEM SOLVING

1	Trace the processing of the input expression $P = (a * b) + (b * c) * 2$ through all phases of a compiler. a) Identify and list all tokens generated during lexical analysis. b) Construct the syntax tree during the parsing phase. c) Generate the intermediate code for the given expression. d) Apply suitable optimizations to the intermediate code. e) Explain the significance of the final target code prod
2	For each of the following Regular Expressions, construct the language. Write any FIVE valid strings for each Regular Expression. a) $(\epsilon + aa^*)^*$ b) $(a^* + b^*)^* abb$ c) $(a^* b^*)^* aba (a^* + b^*)^*$ d) $(a + ab)^*$ e) $(aba)^* + (a^* b^*)^*$
3	i) Find the number of tokens in the following C statement is <pre>main() { int a=10; char b ="abc"; in t c = 30; ch ar d = "xyz"; in /* comment */ t m = 40.5; }</pre> ii) Identify the lexical errors in the following C statement is <pre>void main() { int x=10, y=20; char * a; a= &x; x= 1xab; }</pre>
4	Find a regular expression corresponding to each of the following subsets of $\{0, 1\}^*$: i) The language of all strings that have an even number of 0's. ii) The language of all strings that contain at least one 1. iii) The language of all strings in which both the number of 0's and the number of 1's are odd. iv) The language of all strings that start with 1 and end with 0. v) The language of all strings that do not contain consecutive 1's.

5 Consider a lexical analyzer for a simplified programming language. The language consists of the following tokens:

1. Keywords: if, else, while, return, int, float
2. Operators: +, -, *, /, =, ==, <, >
3. Separators: (,), {, }, :, ,
4. Identifiers: A sequence of letters followed by a sequence of digits, e.g., var123
5. Integer Literals: A sequence of digits, e.g., 12345
6. Floating-point Literals: A sequence of digits followed by a dot and another sequence of digits, e.g., 123.45

The lexical analyzer needs to tokenize the following input string:
int var123 = 12345; float var456 = 123.45; if (var123 == var456) { return; }

Questions:

- a) How many tokens are there in the given input string?
- b) Provide a list of tokens categorized by their types (keywords, operators, separators, identifiers, integer literals, floating-point literals).
- c) Assuming a finite state machine (FSM) is used for lexical analysis, estimate the number of state transitions required to tokenize the given input string. Provide your reasoning based on the transitions needed for each token type.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/for mula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method

2424319

h.v. Ramayogi

CSA-1410SLOT-0COL-ASSESSMENT.

Trace the processing of.

$$P = (a * b) + (b * c) * 2.$$

89/100

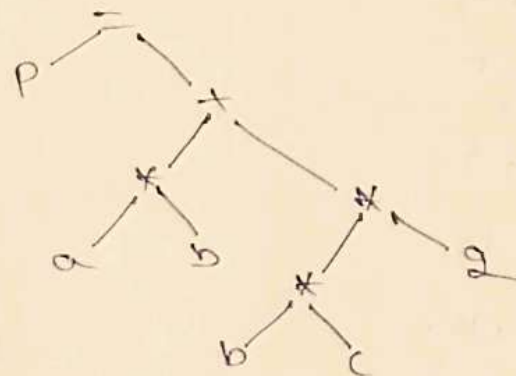
90/100

a) Tokens generated during lexical analysis

Lexeme	Tokens
P	Identifier
=	Assignment operator
(	Left Parenthesis
)	Right Parenthesis
*	multiplication operator
+	Addition operator
2	constant Integer.

b) Syntax tree

$$P = ((a * b) + (b * c) * 2)$$



c, Intermediate code.

$$t_1 = a + b$$

$$t_2 = b * c$$

$$t_3 = t_2 * 2$$

$$t_4 = t_1 + t_3$$

$$P = t_4$$

d, OPTimized code:

$$t_1 = a + b$$

$$t_2 = b * c$$

$$t_3 = t_2 * 2$$

$$P = t_1 + t_3$$

e, Code generator.

MOV R₁, a

MUL R₁, b

MOV R₂, b

MUL R₂, c

MUL R₂, 2

ADD R₁, R₂

MOV P, R₁

Construct the language and give five valid strings.

2(a) $(\epsilon + aa^*)^*$

Since: $aa^* = \{a, aa, aaa, \dots\}$

and ϵ is also allowed, the complete language is $L = \{a\}^*$

valid strings are:

ϵ
a
a a
a a a
a a a a

2(b)

$$(a^* + b^*)^* abb$$

Because $(a^* + b^*)^*$ can generate arbitrary combinations of a and b:

$$(a^* + b^*)^* = (a + b)^*$$

$$\therefore L = (a + b)^* abb$$

valid strings are:

abb
aabb
babb
ababb
bbabb

$$2(c) = (a+b)^* aba (a+b)^*$$

where: $(a^*b)^* = (a+b)^*$

$$\therefore L = (a+b)^* aba (a+b)^*$$

valid strings are:

ab a

a a b a

a b a a

b a b a

a a b a b

$$2(d) = (a+ab)^*$$

Each block can be either: a or ab

valid strings are:

ϵ

a

a a

a b

a a b

$$\text{ex: } aab = a+ab$$

$$2(e) = (aba)^* + (a^*b^*)^*$$

since: $(a^*b^*)^* = (a+b)^*$

the second part already contains all strings over $\{a,b\}$.

$$\therefore L = (a+b)^*$$

valid strings are:

a
b
a b
b a

Lexical Analysis:

i. Number of tokens

Given:

```
main(C)
{
    int a = 10;
    char b = "abc";
    int c = 30;
    char d = "xyz";
    in /* comment */ k m = 40.5;
}
```

<u>Token</u>	<u>Type</u>
main	Identifier
(,), {	Separator
:::	
int	Keyword
a, b, c, d	Identifier
=	Operator
int	Keyword
char	Keyword
"abc"	String Literal
in, k	Identifier / lexeme
40.5	Floating-point Literal

Answer: 33 tokens

ii, Identify lexical errors

Given:

```
void main()
{
    int x=10, y=20;
    char *a;
    a = &x;
    x = 1xab;
}
```

Error 1: $x = 1xab;$

Error 2: $a = \&x;$

Because char^* and x is int , assigning $\&x$ to a creates a type/semantic problem, not a lexical error.

4, Regular expressions over $\{0,1\}^*$

i, Even number of 0s

Regular expression: $1^*(01^*01^*)^*$

EX: $\begin{matrix} \epsilon \\ 1 \\ 11 \\ 00 \\ 1001 \end{matrix}$

iv, Start with 1 and ends with 0

Regular expression = $1(0+1)^*0$

ex: 10

110

100

1010

1110

v, No consecutive 1s.

we can use blocks:

0
10

and optionally finish with 1.

Regular expression = $(0+10)^*(\epsilon+1)$

ex:

ϵ

0

1

10

1010

5, Lexical Analyzer and FSM.

a, Number of tokens

int var123 = 12345;

float var456 = 123.45;

```
if (var123 == var456) {  
    return;  
}
```

ii, At least one 1

Any string over $\{0,1\}$ containing at least one 1

Regular expression: $(0+1)^*1(0+1)^*$

Ex: 1
01
10
101
00100

iii, Both number of 0s and number of 1s are even

$E = (00+11)$

These pairs contribute even parity.

$M = (01+10)$

Each mixed pair contributes one 0 and one 1.

$\therefore$ Regular expression:

$(00+11)^*(01+10)(00+11)^*(01+10)(00+11)^*$

$(01+10)^*(00+11)^*$

$\therefore$ Final valid strings:

01
10
0011
0110
0101

EX: 01, 10, 0011, 010110, 101001

number of tokens = 20.

b) Categorize the tokens

1. keywords

int, float, if, return = 4

2. Operators (==) = 3

2. Separators (,, (,), {, }, ;) = 7

4. Identifiers = 4

5. Integer literals (12345) = 1

6. Floating-point literals

123.45 (1)

verification; $4 + 3 + 7 + 4 + 1 + 1 = 20$

$\therefore$ Total = 20 tokens.

c, FSM state transitions.

• keywords = $3 + 5 + 2 + 6 = 16$

• Identifiers = $6 + 6 + 6 + 6 = 24$

• operators = 4

• separators = 7

• Integer literal = 5 transitions.

$\therefore$ 6 transitions

$\therefore$ Total character-consuming transitions: $16 + 24 + 4 + 7 + 5 + 6 = 62$.

∴ Approach by token-character transitions.